Grid Cell Predictions: Predicting density and assessing factors driving marbled murrelet distribution in Puget Sound using hierarchical distance sampling

Code by S.J. Gillman

Required Packages

```
library(tidyverse)
library(nimble)
library(MCMCvis)
library(sf)
library(matrixStats)
```

Load Data

Required Files Previously Made or Provided

- `Survey_Grid_Info.RData`
- `CH1_nSQ_results_1.RData`
- `CH1_nSQ_results_2.RData`
- `CH1_nSQ_results_3.RData`

Each of the `CH1_nSQ_results_X.RData` contains the posterior samples, the data and constants that went into the model, and the initial values.

```
load("../DataAnalysis/DataFiles/Survey_Grid_Info.RData")
load("RESULTS/CH1_nSQ_results_1.RData")

load("RESULTS/CH1_nSQ_results_2.RData")

load("RESULTS/CH1_nSQ_results_3.RData")



post_pred <- as.matrix(rbind(samples_1[[1]], samples_2[[1]], samples_3[[1]]))

colnames(nimData$DYNAMIC)
# alpha_sea[1] is PAR
# alpha_sea[2] is DO
# alpha_sea[3] is HQH
# alpha_sea[4] is GT
# alpha_sea[5] is DSHORE
# alpha_sea[6] is DSHORESQ

colnames(nimData$STAT_COVS)
# alpha_stat[1] is DEPTH
# alpha_stat[2] is BPI
# alpha_stat[3] is RIE
```

```
# alpha_stat[4] is EI-SAND
# alpha_stat[5] is EI-MIXF
# alpha_stat[6] is EI-BED
# alpha_stat[7] is EI-ART
# alpha_stat[8] is EI-GRAV
# alpha_stat[9] is MI
# alpha_stat[10] is OCN
# alpha_stat[11] is GRAVEL
# alpha_stat[12] is MUD
# alpha_stat[13] is SAND
# alpha_stat[14] is SANDYMUD
# alpha_stat[15] is UMS
# alpha_stat[16] is FUS
# alpha_stat[17] is CUS
# alpha_stat[18] is COBBLE
```

Subset Posterior Samples

```
# For Monthly Estimates
coefs_month <- post_pred %>%
  as.data.frame() %>%
  dplyr::select(c(contains("eps_mon"),
                  contains("alpha_sea"),
                  contains("alpha_stat"),
                  contains("gs.expect")))

# For Seasonal Estimates
coefs_seas <- post_pred %>%
  as.data.frame() %>%
  dplyr::select(c(contains("alpha0"),
                  contains("alpha_sea"),
                  contains("alpha_stat"),
                  contains("gs.expect")))
```

Center and Scale Predictors

```
dynamic_scale <- survey_info %>%
  group_by(month) %>%
  summarise(par_mean = mean(par, na.rm=T),
            par_sd = sd(par, na.rm = T),
            do_mean = mean(do, na.rm=T),
            do_sd = sd(do, na.rm=T),
            gt_mean = mean(grand_range, na.rm=T),
            gt_sd = sd(grand_range, na.rm=T)) %>%
  ungroup()

dynamic_ranges <- survey_info %>%
  group_by(month) %>%
  summarise(par_min = min(par, na.rm=T),
```

```
            par_max = max(par, na.rm = T),
            do_min = min(do, na.rm=T),
            do_max = max(do, na.rm=T),
            gt_min = min(grand_range, na.rm=T),
            gt_max = max(grand_range, na.rm=T)) %>%
  ungroup()



static_scale <- survey_info %>%
  summarise(hab_mean = mean(nest_hab, na.rm=T),
            hab_sd = sd(nest_hab, na.rm=T),
            shore_mean = mean(dist_shore, na.rm=T),
            shore_sd = sd(dist_shore, na.rm=T),
            depth_mean = mean(depth, na.rm=T),
            depth_sd = sd(depth, na.rm=T),
            bpi_mean = mean(bpi, na.rm=T),
            bpi_sd = sd(bpi, na.rm=T),
            rie_mean = mean(rie, na.rm=T),
            rie_sd = sd(rie, na.rm=T))




static_range <- survey_info %>%
  summarise(hab_min = min(nest_hab, na.rm=T),
            hab_max = max(nest_hab, na.rm=T),
            shore_min = min(dist_shore, na.rm=T),
            shore_max = max(dist_shore, na.rm=T),
            depth_min = min(depth, na.rm=T),
            depth_max = max(depth, na.rm=T),
            bpi_min = min(bpi, na.rm=T),
            bpi_max = max(bpi, na.rm=T),
            rie_min = min(rie, na.rm=T),
            rie_max = max(rie, na.rm=T))

static_range2 <- grid_info %>%
  summarise(hab_min = min(nest_hab, na.rm=T),
            hab_max = max(nest_hab, na.rm=T),
            shore_min = min(dist_shore, na.rm=T),
            shore_max = max(dist_shore, na.rm=T),
            depth_min = min(depth, na.rm=T),
            depth_max = max(depth, na.rm=T),
            bpi_min = min(bpi, na.rm=T),
            bpi_max = max(bpi, na.rm=T),
            rie_min = min(rie, na.rm=T),
            rie_max = max(rie, na.rm=T))

grid_substrate <- grid_info %>%
  distinct(ID, seafloor, shoretype) %>%
  mutate(dummy = 1) %>%
 pivot_wider(names_from = shoretype,
       values_from = dummy,
```

```
      values_fill = list(dummy = 0)) %>%
 mutate(cmec_abb = ifelse(
  seafloor == "Coarse Unconsolidated Substrate","CUS",
  ifelse(seafloor == "Fine Unconsolidated Substrate","FUS",
   ifelse(seafloor == "Unconsolidated Mineral Substrate",
      "UMS",gsub(" ","",seafloor))))) %>%
 dplyr::select(-seafloor) %>%
 mutate(dummy = 1) %>%
 pivot_wider(names_from = cmec_abb,
       values_from = dummy,
       values_fill = list(dummy = 0)) %>%
  select(c(ID,
           # shoreline EI-Mix_Coarse is reference
         `EI-Sand`,
         `EI-Mix_Fine`,
         `EI-Boulder`,
         `EI-Artificial`,
         `EI-Gravel`,
         MI,
        OCN,
        # substrate Muddy Sand is reference
         Gravel,
         Mud,
        Sand,
        SandyMud,
         UMS,
         FUS,
        CUS,
         Cobble))


grid_scale <- grid_info %>%
  rename(month = mon) %>%
  full_join(dynamic_scale, by = "month") %>%
  full_join(dynamic_ranges, by = "month") %>%
  # Ensure not above max or min
  mutate(par = ifelse(par > par_max, par_max,
                          ifelse(par < par_min, par_min, par)),
         do = ifelse(do > do_max, do_max,
                     ifelse(do < do_min, do_min, do)),
         grand_range = ifelse(grand_range > gt_max, gt_max,
                     ifelse(grand_range < gt_min, gt_min, grand_range)),
         dist_shore = ifelse(dist_shore > static_range$shore_max,
                             static_range$shore_max,
                     ifelse(dist_shore < static_range$shore_min,
                             static_range$shore_min, dist_shore)),
         nest_hab = ifelse(nest_hab > static_range$hab_max,
                           static_range$hab_max,
                             ifelse(nest_hab< static_range$hab_min,
                                    static_range$hab_min, nest_hab)),
         bpi = ifelse(bpi > static_range$bpi_max, static_range$bpi_max,
                             ifelse(bpi < static_range$bpi_min,
                                    static_range$bpi_min, bpi)),
```

```
         rie = ifelse(rie > static_range$rie_max, static_range$rie_max,
                      ifelse(rie < static_range$rie_min,
                             static_range$rie_min, rie)),
         depth = ifelse(depth > static_range$depth_max,
                        static_range$depth_max,
                      ifelse(depth < static_range$depth_min,
                             static_range$depth_min, depth))) %>%
  mutate(par_scale = (par - par_mean)/par_sd,
         do_scale = (do - do_mean)/do_sd,
         hab_scale = (nest_hab - static_scale$hab_mean)/static_scale$hab_sd,
         gt_scale = (grand_range - gt_mean)/gt_sd,
         shore_scale = (dist_shore - static_scale$shore_mean)/
           static_scale$shore_sd,
         shoreSQ_scale = shore_scale*shore_scale,
         depth_scale = (depth - static_scale$depth_mean)/
           static_scale$depth_sd,
         bpi_scale = (bpi - static_scale$bpi_mean)/
           static_scale$bpi_sd,
         rie_scale = (rie - static_scale$rie_mean)/
           static_scale$rie_sd) %>%
  ## NAs will be filled with averages...
  mutate(par_scale = ifelse(is.na(par_scale), mean(par_scale, na.rm=T),
                            par_scale),
         gt_scale = ifelse(is.na(gt_scale), mean(gt_scale, na.rm=T),
                           gt_scale),
         do_scale = ifelse(is.na(do_scale), mean(do_scale, na.rm=T),
                           do_scale)) %>%
  select(c(ID, numid, seasNum,month, mNum, year, myNum,
           par_scale, do_scale, hab_scale,
           gt_scale, shore_scale, shoreSQ_scale, depth_scale,
           bpi_scale, rie_scale)) %>%
  full_join(grid_substrate, by = "ID") %>%
  full_join(grid_info %>%
              distinct(ID, area), by = "ID") %>%
  arrange(year, mNum, numid)

sapply(grid_scale, function(x) sum(is.na(x)))

total_area <- grid_info %>%
  distinct(ID, area) %>%
  summarise(totarea = sum(area)) %>%
  pull(totarea)


total_area
```

Make Functions for Density Predictions

Month-Year

```
predict_density <- function(target_month, target_year,
                            grid_scale, coefs_month) {
```

```r
  # 6 dynamic environmental variables (change by season index)
  dynamic_covs <- c("par_scale", "do_scale", "hab_scale",
                    "gt_scale", "shore_scale", "shoreSQ_scale")

# 18 static environmental variables (constant across seasons)
  static_covs <- c("depth_scale", "bpi_scale", "rie_scale",
                   "EI-Sand", "EI-Mix_Fine",
                  "EI-Boulder", "EI-Artificial", "EI-Gravel",
                  "MI", "OCN", "Gravel",
                  "Mud", "Sand", "SandyMud", "UMS",
                  "FUS", "CUS", "Cobble")

  # Subset grid data for this specific month-year chunk
  df_sub <- grid_scale %>%
    filter(month == target_month, year == target_year) %>%
    arrange(numid)

  G <- nrow(df_sub)
  I <- nrow(coefs_month)
  if(G == 0) return(NULL) # Skip if no grid data matches

  # Convert grid covariates into design matrices
  DYNAMIC <- as.matrix(df_sub[, dynamic_covs])
  STAT_COVS  <- as.matrix(df_sub[, static_covs])

  # Pull model indices unique to this month-year chunk
  mID <- unique(df_sub$mNum)
  sID <- unique(df_sub$seasNum)
  myID <- unique(df_sub$myNum)

  # Extract posterior parameters using explicit naming
  eps_mon <- coefs_month[[paste0("eps_mon[", mID, "]")]]
  gs_expected <- coefs_month[[paste0("gs.expected[", myID, "]")]]

  sea_cols <- paste0("alpha_sea[", sID, ", ", 1:6, "]")
  alpha_sea <- as.matrix(coefs_month[, sea_cols])

  stat_cols <- paste0("alpha_stat[", 1:18, "]")
  alpha_stat <- as.matrix(coefs_month[, stat_cols])

  # Multiply
  dynamic_coefs <- DYNAMIC %*% t(alpha_sea)
  static_coefs <- STAT_COVS %*% t(alpha_stat)

  # Combine components into log lambda space
  log_group <- matrix(eps_mon, nrow = G, ncol = I, byrow = TRUE) +
                   dynamic_coefs +
                   static_coefs

  # Group Size
  est_lambda <- exp(log_group) * matrix(gs_expected, nrow = G,
                                        ncol = I, byrow = TRUE)
```

```
  # Summarize across posterior draws
  summary_df <- data.frame(
    numid = df_sub$numid,
    month = target_month,
    year = target_year,
    AVE = rowMeans(est_lambda, na.rm = TRUE),
    MED = apply(est_lambda, 1, median, na.rm = TRUE),
    LCI = apply(est_lambda, 1, quantile, probs = 0.025, na.rm = TRUE),
    Q25 = apply(est_lambda, 1, quantile, probs = 0.25, na.rm = TRUE),
    Q75 = apply(est_lambda, 1, quantile, probs = 0.75, na.rm = TRUE),
    UCI = apply(est_lambda, 1, quantile, probs = 0.975, na.rm = TRUE),
    SD = apply(est_lambda, 1, sd, na.rm = TRUE)) %>%
    mutate(CV = SD / AVE, IQR = Q75-Q25)

  # Return summary statistics, raw chains for averaging later, and cell metadata
  return(list(summary = summary_df, raw_posterior = est_lambda,
              metadata = df_sub[, c("numid", "month", "year", "area")]))
}
```

Average Month

```
month_ave <- function(target_month, mo_yr_grid, all_months) {

  # Find which list positions match your target month across all years
  matching_indices <- which(mo_yr_grid$month == target_month)

  if(length(matching_indices) == 0) return(NULL)

  # Extract matrices and layout templates
  raw_matrices <- map(matching_indices, ~ all_months[[.x]]$raw_posterior)
  pooled_posterior <- do.call(cbind, raw_matrices)

  # Build long-term summary
  long_term_summary <- data.frame(
    numid = all_months[[matching_indices[1]]]$metadata$numid, # grid structure
    month = target_month,
    AVE = rowMeans(pooled_posterior, na.rm = TRUE),
    MED = apply(pooled_posterior, 1, median, na.rm = TRUE),
    LCI = apply(pooled_posterior, 1, quantile, probs = 0.025, na.rm = TRUE),
    Q25 = apply(pooled_posterior, 1, quantile, probs = 0.25, na.rm = TRUE),
    Q75 = apply(pooled_posterior, 1, quantile, probs = 0.75, na.rm = TRUE),
    UCI = apply(pooled_posterior, 1, quantile, probs = 0.975, na.rm = TRUE),
    SD = apply(pooled_posterior, 1, sd, na.rm = TRUE)) %>%
    mutate(CV = SD / AVE, IQR = Q75 - Q25)

  return(long_term_summary)
}
```

Global Month-Year

```
overall_month_year <- function(target_month, target_year, all_months) {

  match_idx <- keep(all_months, ~ .x$metadata$month[1] == target_month &&
                                  .x$metadata$year[1] == target_year)

  if (length(match_idx) == 0) return(NULL)

  #  Extract
  raw_mat <- match_idx[[1]]$raw_posterior

  cell_areas <- as.numeric(match_idx[[1]]$metadata$area)

  # Flatten
  abundance_matrix <- raw_mat * cell_areas

  total_abund <- colSums(abundance_matrix, na.rm = TRUE)

  abund_summary <- data.frame(
    month = target_month,
    year = target_year,
    AVE = mean(total_abund, na.rm = TRUE),
    MED = median(total_abund, na.rm = TRUE),
    LCI = quantile(total_abund, probs = 0.025, na.rm = TRUE),
    Q25 = quantile(total_abund, probs = 0.25, na.rm = TRUE),
    Q75 = quantile(total_abund, probs = 0.75, na.rm = TRUE),
    UCI = quantile(total_abund, probs = 0.975, na.rm = TRUE),
    SD = sd(total_abund, na.rm = TRUE)) %>%
    mutate(CV = SD / AVE, IQR = Q75 - Q25)

  total_area <- sum(cell_areas)

  total_den <- total_abund/total_area

  overall_summary <- data.frame(
    month = target_month,
    year = target_year,
    AVE = mean(total_den, na.rm = TRUE),
    MED = median(total_den, na.rm = TRUE),
    LCI = quantile(total_den, probs = 0.025, na.rm = TRUE),
    Q25 = quantile(total_den, probs = 0.25, na.rm = TRUE),
    Q75 = quantile(total_den, probs = 0.75, na.rm = TRUE),
    UCI = quantile(total_den, probs = 0.975, na.rm = TRUE),
    SD = sd(total_den, na.rm = TRUE)) %>%
    mutate(CV = SD / AVE, IQR = Q75 - Q25)

  rownames(abund_summary)   <- NULL
  rownames(overall_summary) <- NULL

  return(list(densities = overall_summary, abundance = abund_summary))
}
```

Global Month

```
month_ave_global <- function(target_month, mo_yr_grid, all_months) {

  # Find which list positions match your target month across all years
  matching_indices <- which(mo_yr_grid$month == target_month)

  if(length(matching_indices) == 0) return(NULL)

  # Extract matrices and layout templates
  raw_matrices <- map(matching_indices, ~ all_months[[.x]]$raw_posterior)
  pooled_posterior <- do.call(cbind, raw_matrices)

  # Flatten the entire matrix
  flat_posterior <- as.vector(pooled_posterior)

  long_term_summary <- data.frame(
    month = target_month,
    AVE = mean(flat_posterior, na.rm = TRUE),
    MED = median(flat_posterior, na.rm = TRUE),
    LCI = quantile(flat_posterior, probs = 0.025, na.rm = TRUE),
    Q25 = quantile(flat_posterior, probs = 0.25, na.rm = TRUE),
    Q75 = quantile(flat_posterior, probs = 0.75, na.rm = TRUE),
    UCI = quantile(flat_posterior, probs = 0.975, na.rm = TRUE),
    SD  = sd(flat_posterior, na.rm = TRUE)) %>%
    mutate(CV = SD / AVE, IQR = Q75 - Q25)

  # Clean up row names from the quantile outputs
  rownames(long_term_summary) <- NULL

  return(long_term_summary)
}
```

Season-Year

```
predict_density_SEASON <- function(target_season, target_year,
                                   grid_scale, coefs_seas) {

  # 6 dynamic environmental variables
  dynamic_covs <- c("par_scale", "do_scale", "hab_scale",
                    "gt_scale", "shore_scale", "shoreSQ_scale")

# 18 static environmental variables
  static_covs  <- c("depth_scale", "bpi_scale", "rie_scale",
                    "EI-Sand", "EI-Mix_Fine",
                  "EI-Boulder", "EI-Artificial", "EI-Gravel",
                  "MI", "OCN", "Gravel",
                  "Mud", "Sand", "SandyMud", "UMS",
                  "FUS", "CUS", "Cobble")

  # Subset grid data for this specific season chunk
```

```r
  df_sub <- grid_scale %>%
    filter(seasNum == target_season & year == target_year) %>%
    arrange(numid)

  # Pull model indices unique to this season chunk
  sID <- target_season
  gID <- sort(unique(df_sub$numid)) # can have more than one occurance
  myID <- unique(df_sub$myNum) # myNum is distinct 1-100

  G <- length(unique(gID)) # 1545
  MY <- length(unique(myID)) # 100
  I <- nrow(coefs_seas) # 15000

  if(G == 0) return(NULL) # some years don't have both seasons

  # Season Specific intercept
  alpha0 <- coefs_seas[[paste0("alpha0[", sID, "]")]]

  # dynamic covariates
  sea_cols <- paste0("alpha_sea[", sID, ", ", 1:6, "]")
  alpha_sea <- as.matrix(coefs_seas[, sea_cols])

  # static covariates
  stat_cols <- paste0("alpha_stat[", 1:18, "]")
  alpha_stat <- as.matrix(coefs_seas[, stat_cols])

  # temp matrix
  month_matrices <- list()

  for (m in seq_along(myID)){
    # subset to just one month
    df_month <- df_sub %>%
      filter(myNum == myID[m]) %>%
      arrange(numid)

    if(nrow(df_month) == 0) next
    # Convert grid covariates to matrices
    DYNAMIC <- as.matrix(df_month[, dynamic_covs])
    STAT_COVS <- as.matrix(df_month[, static_covs])

    # set-up for group size
    gs_matrix <- matrix(0, nrow = G, ncol = I)
    for(g in 1:G) {
      col_gs <- paste0("gs.expected[", df_month$myNum[g], "]")
      gs_matrix[g, ] <- coefs_seas[[col_gs]]
    }
# Compute log linear predictors
    dynamic_coefs <- DYNAMIC %*% t(alpha_sea)
    static_coefs  <- STAT_COVS %*% t(alpha_stat)

    log_group <- matrix(alpha0, nrow = G, ncol = I, byrow = T) +
                 dynamic_coefs +
                 static_coefs
```

```
    # Final total Density for this month
    month_matrices[[m]] <- exp(log_group) * gs_matrix
  }

  # Pool all grid 1 season 1 together regardless of month for this year
  pooled_posterior <- do.call(cbind, month_matrices)

  # Try to free up space...
  rm(month_matrices)
  gc()

  # Quantiles
  probs <- c(0.025, 0.25, 0.75, 0.975)
  quants <- rowQuantiles(pooled_posterior, probs = probs, na.rm = T)

  summary_df <- data.frame(
    numid = gID,
    seasNum = target_season,
    year = target_year,
    AVE = rowMeans(pooled_posterior, na.rm = T),
    MED = rowMedians(pooled_posterior, na.rm = T),
    LCI = quants[, "2.5%"],
    Q25 = quants[, "25%"],
    Q75 = quants[, "75%"],
    UCI = quants[, "97.5%"],
    SD = rowSds(pooled_posterior, na.rm = T)) %>%
    mutate(CV = SD / AVE, IQR = Q75 - Q25)

  return(list(
    summary = summary_df,
    raw_posterior = pooled_posterior,
    metadata = data.frame(numid = gID, seasNum = target_season,
                          year = target_year)
  ))
}
```

Average Season

```
season_ave <- function(target_season, sea_yr_grid, all_seasons) {

  # season indices
  matching_indices <- which(sea_yr_grid$seasNum == target_season)
  if(length(matching_indices) == 0) return(NULL)

  unique_ids <- unique(unlist(lapply(matching_indices, function(idx) {
    all_seasons[[idx]]$metadata$numid
  })))

  n_unique <- length(unique_ids)

  # flat vectors
```

```r
ave_val <- numeric(n_unique)
med_val <- numeric(n_unique)
lci <- numeric(n_unique)
q25 <- numeric(n_unique)
q75 <- numeric(n_unique)
uci <- numeric(n_unique)
sd_val <- numeric(n_unique)

# Loop over unique IDs
for (j in seq_len(n_unique)) {
  current_id <- unique_ids[j]
  pooled_draws <- numeric(0)

  for (idx in matching_indices) {
    grid_ids <- all_seasons[[idx]]$metadata$numid
    match_rows <- which(grid_ids == current_id)

    if (length(match_rows) > 0) {

      mat_sub <- all_seasons[[idx]]$raw_posterior[match_rows, , drop = F]

      pooled_draws <- c(pooled_draws, as.vector(mat_sub))
    }
  }


  if (length(pooled_draws) == 0 || all(is.na(pooled_draws))) {
    ave_val[j] <- NA; med_val[j] <- NA; lci[j] <- NA; q25[j] <- NA
    q75[j] <- NA; uci[j] <- NA; sd_val[j] <- NA
    next
  }

  quants <- quantile(pooled_draws, probs = c(0.025, 0.25, 0.50,
                                             0.75, 0.975), na.rm = T)
  ave_val[j] <- mean(pooled_draws, na.rm = T)
  med_val[j] <- quants[3]
  lci[j] <- quants[1]
  q25[j] <- quants[2]
  q75[j] <- quants[4]
  uci[j] <- quants[5]
  sd_val[j] <- sd(pooled_draws, na.rm = T)
}

# Build the final output data frame all at once
long_term_summary <- data.frame(
  numid = unique_ids,
  seasNum = target_season,
  AVE = ave_val,
  MED = med_val,
  LCI = lci,
  Q25 = q25,
  Q75 = q75,
  UCI = uci,
```

```
    SD = sd_val,
    CV = sd_val / ave_val,
    IQR = q75 - q25)

  return(long_term_summary)
}
```

Prediction Monthly

Month-Year

```
mo_yr_grid <- grid_scale %>% distinct(month, year)

# Run the matrix prediction across all combinations
all_months <- purrr::map2(mo_yr_grid$month, mo_yr_grid$year, function(m, y) {
  message(paste("Predicting Month-Year:", m, y))
  predict_density(m, y, grid_scale, coefs_month)
})

# Combine all
predicted_month_years <- map_df(all_months, ~ .x$summary)
```

```
saveRDS(predicted_month_years, file = "RESULTS/predicted_month_years.rds")
```

Monthly Average

```
unique_months <- unique(mo_yr_grid$month)

Monthly_averages <- map_df(unique_months, function(m) {
  message(paste("Calculating Average for:", m))
  month_ave(target_month = m, mo_yr_grid = mo_yr_grid, all_months = all_months)
})
```

```
saveRDS(Monthly_averages, file = "RESULTS/predicted_months.rds")
```

Global Month-Year Average

```
month_year_global <- purrr::map2(mo_yr_grid$month,
                                 mo_yr_grid$year, function(m, y) {
  message(paste("Predicting Month-Year:", m, y))
  overall_month_year(m, y, all_months)
})
```

```
my_global_df <- map_df(month_year_global, ~ .x$densities)
myA_global_df <- map_df(month_year_global, ~ .x$abundance)
saveRDS(my_global_df, file = "RESULTS/predicted_month_year_global.rds")

saveRDS(myA_global_df, file = "RESULTS/predictedA_month_year_global.rds")
```

Global Monthly Average

```
Monthly_global <- map_df(unique_months, function(m) {
  message(paste("Calculating Average for:", m))
  month_ave_global(target_month = m, mo_yr_grid = mo_yr_grid,
                   all_months = all_months)
})
```

```
saveRDS(Monthly_global, file = "RESULTS/predicted_months_global.rds")
```

```
rm(all_months)
rm(samples_1)
rm(samples_2)
rm(samples_3)
rm(coefs_month)
rm(init_1)
rm(init_2)
rm(init_3)
rm(post_pred)
gc()
```

Prediction Seasonal

Season-Year

```
sea_yr_grid <- grid_scale %>% distinct(seasNum, year)

# Run the matrix prediction across all combinations
all_seasons <- purrr::map2(sea_yr_grid$seasNum,
                           sea_yr_grid$year, function(s, y) {
  message(paste("Predicting Season-Year:", s, y))
  predict_density_SEASON(s, y, grid_scale, coefs_seas)
})

# Combine all
predicted_season_years <- map_df(all_seasons, ~ .x$summary)
```

```
saveRDS(predicted_season_years, file = "RESULTS/predicted_season_years.rds")
```

Seasonal Average

```
unique_seasons <- unique(sea_yr_grid$seasNum)

Seasonal_averages <- map_df(unique_seasons, function(s) {
  message(paste("Calculating Average for:", s))
  season_ave(target_season = s, sea_yr_grid = sea_yr_grid,
             all_seasons = all_seasons3)
```

```
})

Breeding_averages <- season_ave(target_season = 1,
                                sea_yr_grid = sea_yr_grid,
                                all_seasons = all_seasons)

NBreeding_averages <- season_ave(target_season = 2,
                                 sea_yr_grid = sea_yr_grid,
                                 all_seasons = all_seasons)
saveRDS(NBreeding_averages, file = "RESULTS/predicted_seasonsNB.rds")
```

```
saveRDS(Breeding_averages, file = "RESULTS/predicted_seasonsB.rds")
saveRDS(NBreeding_averages, file = "RESULTS/predicted_seasonsNB.rds")
rm(all_seasons2)
```